

Système d'alertes & notifications

Conception et implémentation — Optimi Santé (suite du Sprint 5).

Mis à jour le 4 septembre 2026 • Périmètre : notifications in-app multi-rôles, e-mails transactionnels, alertes opérationnelles admin • État : **Phases 1 & 2 livrées** — déduplication/regroupement, accusé de réception, mise en veille, seuils configurables en base, préférences par utilisateur, écran admin « Alertes ». Reste : SSE temps réel, Web Push, e-mails digest (\$7).

1. Réponse à ta question : pour qui ?

Oui — les notifications concernent tous les acteurs de la plateforme, pas seulement l'admin. Chaque rôle reçoit ce qui le concerne :

Rôle	Exemples de notifications reçues
Client B2C (CLIENT_B2C)	Paielement de commande confirmé + reçu ; (à venir) expédition, statut de candidature médecin, export RGPD prêt.
Entreprise B2B (CLIENT_B2B)	Paielement / statut de commande & devis ; (à venir) facture, avancement des dossiers des médecins sponsorisés.
Médecin (MEDECIN)	Compte activé ; chaque changement de statut de son dossier de formation (validation académique, administrative, convention émise, visa...) ; (à venir) session affectée, documents prêts.
Partenaire / CHU (CENTRE_FORMATION)	Compte partenaire activé ; (à venir) nouveau dossier à examiner, capacité de session atteinte.
Admin (ADMIN)	Alertes opérationnelles : nouvelle demande de partenariat, nouvelle candidature médecin, rapports batch en échec ; (à venir) paielement Stripe échoué, stock bas.
Super Admin (SUPER_ADMIN)	Reçoit tout ce que reçoit l'admin ; (phase 2) alertes de gouvernance : pic d'anonymisations RGPD, anomalies du journal d'audit.

Techniquement : une notification vise soit **un utilisateur précis** (notifyUser), soit **un rôle entier** (notifyRole). Les notifications de rôle sont **dépliées en une ligne par destinataire** au moment de l'écriture, pour que l'état lu / non lu reste individuel.

2. Les 4 canaux (et ce qui est livré)

#	Canal	Rôle	Phase 1
1	Toasts	Retour d'action immédiat dans l'UI (« enregistré », « erreur »). Purement front.	✔ ToastProvider global + useToast()
2	Notifications in-app	Cloche 🔔 + compteur non-lus + liste. Persistées, historisées, par utilisateur.	✔ table + REST + cloche (polling 30 s)
3	E-mails transactionnels	Confirmation de paielement, évolution de dossier... via EmailService (Mailtrap).	✔ sendHtml() + charte commune, câblé sur 2 événements
4	Alertes opérationnelles	Veille par seuils → notification de rôle + e-mail. Jobs @Scheduled .	✔ 1 job livré (rapports en échec), pattern posé

3. Architecture backend

Le nouveau domaine `domain/notification/` est **calqué sur** `domain/audit/` (livré au Sprint 5) : entité + writer isolé en transaction indépendante + service *fail-soft*.

3.1 Le principe : événements de domaine, pas d'appels directs

Avant, un service métier appelait `emailService` en dur. Désormais il **publie un événement** ; un écouteur **APRÈS COMMIT** et **asynchrone** fait le fan-out vers les canaux. La logique de notification sort des services métier et devient *fail-soft* par construction (hors transaction critique, hors chemin de réponse).

Service métier (transaction)

order.setPaymentStatus(PAID) → save

eventPublisher.publishEvent(OrderPaid(...))

▼ COMMIT

NotificationDispatcher @TransactionalEventListener(AFTER_COMMIT) @Async

↳ NotificationService.notifyUser(...) → NotificationWriter (REQUIRES_NEW) → INSERT notifications

↳ EmailService.sendHtml(...) → SMTP (Mailtrap), échec avalé

3.2 Fichiers créés

Fichier	Lignes	Rôle
domain/notification/entity/Notification.java	~78	Entité <code>notifications</code> : <code>tenantId</code> , <code>recipientUserId</code> , <code>type</code> , <code>severity</code> (INFO/SUCCESS/WARNING/CRITICAL), <code>title</code> , <code>body</code> (TEXT), <code>linkUrl</code> , <code>metadata</code> (TEXT), <code>readAt</code> , <code>createdAt</code> . <code>@PrePersist</code> pour les défauts.
domain/notification/entity/NotificationSeverity.java	12	Enum de niveau (icône + couleur côté front, filtre des alertes).
domain/notification/repository/NotificationRepository.java	~35	Requêtes bornées au destinataire : page « mes notifs », page « non lues », <code>countByRecipientUserIdAndReadAtIsNull</code> , <code>markAllRead</code> . Paramètres toujours non nuls → pas de piège JPQL null sur PostgreSQL.
domain/notification/service/NotificationWriter.java	~38	Persistance isolée, <code>@Transactional(REQUIRES_NEW)</code> , exceptions avalées (jumeau de <code>AuditLogWriter</code>).
domain/notification/service/NotificationService.java	~85	<code>notifyUser(...)</code> et <code>notifyRole(...)</code> (dépliage par utilisateur, plafond 500). Résout le tenant depuis le contexte, sinon depuis le destinataire (<code>UserRepository.findTenantIdById</code>). Ne lève jamais.
domain/notification/event/NotificationEvents.java	~55	6 record d'événements de domaine : <code>OrderPaid</code> , <code>EnrollmentStatusChanged</code> , <code>DoctorAccountValidated</code> , <code>PartnerAccountValidated</code> , <code>PartnershipRequestSubmitted</code> , <code>DoctorApplicationSubmitted</code> .
domain/notification/service/NotificationDispatcher.java	~95	6 écouteurs <code>@Async @TransactionalEventListener(AFTER_COMMIT)</code> : chaque événement → notification in-app (+ e-mail pour <code>OrderPaid</code> et <code>EnrollmentStatusChanged</code>).
domain/notification/service/NotificationAlertJob.java	~48	Alerte planifiée : chaque matin 07:30, si des <code>report_runs</code> ont échoué sur 24 h → notification CRITICAL au rôle ADMIN. Illustre le canal « veille par seuils ».
domain/notification/dto/NotificationDto.java	~40	DTO de sortie + fabrique <code>from(Notification)</code> (expose <code>read = readAt != null</code>).
domain/notification/web/NotificationResource.java	~75	<code>/api/v1/notifications</code> : GET "" (page, size, unreadOnly), GET /unread-count, POST /{id}/read, POST /read-all. Destinataire = principal JWT. Sécurité : <code>anyRequest().authenticated()</code> (déjà en place).
resources/db/migration/V23__notifications.sql	~35	Table <code>notifications</code> + index (<code>recipient_user_id</code> , <code>created_at</code> DESC) et index partiel <code>WHERE read_at IS NULL</code> (compteur de pastille). <code>metadata</code> en TEXT (cohérent V22/V23).

3.3 Fichiers modifiés

Fichier	Modification
config/SchedulingConfig.java	Ajout de <code>@EnableAsync</code> (les écouteurs de notification s'exécutent hors du thread de la requête).

common/email/EmailService.java	Ajout de <code>sendHtml(to, subject, heading, innerHtml) + wrap(...)</code> : charte visuelle commune (en-tête vert / pied de page) réutilisable. <code>sendCredentialsEmail</code> inchangé. Toujours <i>fail-soft</i> .
domain/identity/repository/UserRepository.java	Ajout de <code>findTenantIdById(UUID)</code> (identifiant de tenant sans charger l'entité LAZY).
domain/orders/service/OrderService.java	<code>confirmOrderPayment</code> : publie <code>OrderPaid</code> après déduction de stock.
domain/training/service/EnrollmentService.java	<code>updateEnrollmentStatus</code> et <code>reviewAcademic</code> : publient <code>EnrollmentStatusChanged</code> (ancien → nouveau statut), uniquement si le statut change réellement.
domain/doctorapplication/service/DoctorApplicationService.java	<code>submitApplication</code> → <code>DoctorApplicationSubmitted</code> (alerte admin) ; <code>confirmPayment</code> → <code>DoctorAccountValidated</code> (bienvenue médecin).
domain/partnership/service/PartnershipService.java	<code>submitRequest</code> → <code>PartnershipRequestSubmitted</code> (alerte admin) ; <code>approveRequest</code> → <code>PartnerAccountValidated</code> .

3.4 Correspondance événement → notification

Événement	Déclencheur	Destinataire	In-app	E-mail
<code>OrderPaid</code>	Webhook Stripe / confirmation paiement	Acheteur	SUCCESS « Paiement confirmé »	Oui
<code>EnrollmentStatusChanged</code>	Admin ou partenaire change le statut d'un dossier	Médecin du dossier	INFO « Dossier mis à jour »	Oui
<code>DoctorAccountValidated</code>	Candidature payée → compte créé	Le médecin	SUCCESS « Compte activé »	— (déjà l'e-mail d'identifiants)
<code>PartnerAccountValidated</code>	Demande de partenariat approuvée	Le partenaire	SUCCESS « Compte activé »	— (déjà l'e-mail d'identifiants)
<code>PartnershipRequestSubmitted</code>	Formulaire « Devenir partenaire »	Rôle ADMIN	WARNING « Nouvelle demande »	—
<code>DoctorApplicationSubmitted</code>	Formulaire de candidature médecin	Rôle ADMIN	INFO « Nouvelle candidature »	—
<code>REPORT_FAILURE</code> (job)	Cron 07:30 — <code>report_runs</code> FAILED sur 24 h	Rôle ADMIN	CRITICAL « Rapports en échec »	—

4. Architecture frontend

4.1 Fichiers créés

Fichier	Lignes	Rôle
src/context/ToastContext.tsx	~95	<code>ToastProvider</code> + <code>useToast()</code> : pile de toasts globale en bas à droite (success / error / info / warning), auto-disparition 5 s. Remplace le state <code>Toast</code> dupliqué page par page.
src/api/notificationService.ts	~50	Client API : <code>list(page, size, unreadOnly)</code> , <code>unreadCount()</code> , <code>markRead(id)</code> , <code>markAllRead()</code> + types <code>AppNotification</code> , <code>Page<T></code> .
src/hooks/useNotifications.ts	~45	<code>useUnreadCount()</code> (react-query, <code>refetchInterval: 30 s</code> , actif seulement si token présent), <code>useNotificationList(open)</code> (chargée à l'ouverture du panneau), <code>useNotificationActions()</code> (mutations + invalidation).
src/components/common/NotificationBell.tsx	~120	Cloche + pastille rouge (99+), panneau déroulant : liste des 15 dernières, pastille de couleur par <code>severity</code> , temps relatif (« il y a 3 min »), clic → <code>navigate(linkUrl)</code> + marque lu, bouton « Tout marquer lu », fermeture au clic extérieur.

4.2 Fichiers modifiés

Fichier	Modification
src/main.tsx	<code><ToastProvider></code> ajouté autour de <code><App/></code> (sous <code>QueryClientProvider</code> / <code>AuthProvider</code>).
src/layouts/AdminLayout.tsx	Barre supérieure fine (sticky) dans <code><main></code> avec <code><NotificationBell/></code> .
src/layouts/DoctorLayout.tsx	Idem — la cloche apparaît dans l'espace médecin.
src/layouts/PartnerLayout.tsx	Idem — la cloche apparaît dans l'espace partenaire.

Note temps réel : Phase 1 = interrogation périodique (30 s) via react-query. Pas de dépendance nouvelle. Le passage en SSE (`GET /notifications/stream`, `EventSource`) est prévu en phase 2 — il demande de gérer l'auth du flux (l' `EventSource` ne peut pas poser l'en-tête `Authorization`).

5. Vérifications

Élément	Résultat
Compilation backend (<code>mvnw compile</code> , JDK 24 → cible <code>release 21</code>)	✅ BUILD SUCCESS , 0 erreur — 6 écouteurs <code>@TransactionalEventListener</code> + <code>@EnableAsync</code> chargés sans incident au démarrage.
Build frontend (<code>tsc -b</code> + <code>vite build</code>)	✅ 2579 modules, build en 7,9 s, 0 erreur TypeScript.
Image Docker backend reconstruite + conteneur relancé	✅ <code>docker compose build backend</code> (exit 0) puis <code>up -d</code> — « Started BackendApplication ».
Migration Flyway <code>V23__notifications.sql</code>	✅ log au démarrage : « <i>Migrating schema to version 23 - notifications ... Successfully applied 1 migration, now at version v23</i> ».
Endpoints <code>/api/v1/notifications/*</code>	✅ <code>GET /unread-count</code> 2 → 1 → 0 ; <code>GET ""</code> renvoie sévérité / type / <code>read</code> corrects et uniquement les notifications du destinataire ; <code>POST /{id}/read</code> → 204 ; <code>POST /read-all</code> → {updated: 1}.
Fan-out d'événement bout-en-bout	✅ <code>PATCH /admin/orders/{id}/confirm-payment</code> → 200 : après commit, l'écouteur async a produit chez l'acheteur une notification SUCCESS « Paiement confirmé » (n° de commande + montant + lien <code>/dashboard/orders</code>). Chaîne vérifiée : <code>publish</code> → <code>listener AFTER_COMMIT @Async</code> → <code>NotificationService</code> → <code>NotificationWriter (REQUIRES_NEW)</code> → <code>INSERT</code> → <i>API de lecture</i> → <i>bon destinataire</i> .

Les déclencheurs qui passent par Cloudinary / Stripe / S3 (soumission de partenariat, candidature médecin) n'ont pas pu être exercés par HTTP dans cet environnement local — ces services externes ne sont pas configurés, comme déjà noté pour d'autres fonctions au Sprint 5. Le câblage de ces événements est vérifié à la compilation et au démarrage ; le chemin complet est prouvé par le test `OrderPaid` ci-dessus (mécanisme identique). Données de test nettoyées après vérification (table `notifications` vide).

6. Phase 2 — livrée

Migration `V24__notifications_phase2.sql` : colonnes `dedupe_key`, `group_count`, `acknowledged_at`, `acknowledged_by`, `snoozed_until` sur `notifications` ; tables `alert_thresholds` (3 sondes semées) et `notification_preferences`.

6.1 Déduplication / regroupement

Chaque notification peut porter une `dedupeKey`. À l'écriture (`NotificationWriter.persistOrGroup`), s'il existe déjà une notification **non acquittée** de même clé pour le même destinataire : au lieu d'empiler un doublon, on **incrémente** `group_count`, on rafraîchit le titre / corps / sévérité, on la repasse non lue et on la fait remonter en tête. Clés utilisées : `ORDER_PAID:<orderId>` (webhook Stripe rejoué = pas de doublon), `ENROLLMENT_STATUS:<enrollmentId>`, et la clé de sonde pour les alertes de veille. Le front affiche `×N`.

6.2 Accusé de réception & mise en veille

Action	Endpoint	Effet
Acquitter	POST /api/v1/notifications/{id}/ack	<code>acknowledged_at</code> + <code>acknowledged_by</code> = utilisateur courant, marque lu. Sort la ligne des « alertes ouvertes » et la rend de nouveau déduplicable à neuf.
Mettre en veille	POST /api/v1/notifications/{id}/snooze?hours=24	<code>snoozed_until</code> = maintenant + N h (max 720). La notification est masquée de la liste et du compteur jusqu'à échéance.

6.3 Seuils de veille configurables (fin des constantes en dur)

`NotificationAlertJob` tourne désormais **toutes les heures** et lit `alert_thresholds` : pour chaque sonde activée, il exécute la requête correspondante sur la `window_hours` configurée et, si le compte $\geq$ `threshold_value`, pousse une alerte au rôle ADMIN à la `severity` configurée (déduplicquée par clé de sonde). Sondes livrées : `REPORT_FAILURE` (rapports du worker en échec), `ORDER_UNPAID_STALE` (commandes impayées anciennes), `ENROLLMENT_STALE` (dossiers CHU en attente d'examen). Édition via `GET/PUT /api/v1/admin/alerts/thresholds`.

6.4 Préférences par utilisateur

Table `notification_preferences` (`user_id`, `type`) $\rightarrow$ `in_app_enabled` / `email_enabled`. **Absence de ligne = tout activé.** `NotificationService` vérifie `in_app_enabled` avant d'écrire ; `NotificationDispatcher` vérifie `email_enabled` avant l'e-mail. API `GET /api/v1/notifications/preferences` + `PUT /api/v1/notifications/preferences/{type}`. Écran `/notifications/settings` (grille type $\times$ In-app $\times$ E-mail).

6.5 Écran admin « Alertes »

`AdminAlertResource` + `AlertQueryService` (SQL natif filtré, borné à l'admin courant). Page `/admin/alerts` (entrée de menu **Alertes**) : tableau filtrable par **sévérité** (Alertes = Warning+Critical / Critical / Warning / Toutes) et **statut** (Ouvertes / Acquittées / Toutes), actions Acquitter & Mettre en veille par ligne, ouverture de l'écran concerné, et panneau d'édition des seuils de veille.

6.6 Nouveaux fichiers phase 2

Backend	Frontend
<code>entity/AlertThreshold.java</code> , <code>entity/NotificationPreference.java</code> <code>repository/AlertThresholdRepository.java</code> , <code>repository/NotificationPreferenceRepository.java</code> <code>service/AlertQueryService.java</code> <code>web/AdminAlertResource.java</code> <code>db/migration/V24__notifications_phase2.sql</code> <i>modifiés</i> : <code>Notification</code> , <code>NotificationRepository</code> , <code>NotificationWriter</code> , <code>NotificationService</code> , <code>NotificationDispatcher</code> , <code>NotificationAlertJob</code> , <code>NotificationResource</code> , <code>NotificationDto</code>	<code>api/adminAlertService.ts</code> <code>pages/NotificationsPage.tsx</code> (/notifications) <code>pages/NotificationSettingsPage.tsx</code> (/notifications/settings) <code>pages/admin/AdminAlertsPage.tsx</code> (/admin/alerts) <i>modifiés</i> : <code>api/notificationService.ts</code> , <code>hooks/useNotifications.ts</code> , <code>components/common/NotificationBell.tsx</code> ($\times N$ + liens), <code>components/Navbar.tsx</code> (cloche boutique), <code>App.tsx</code> , <code>layouts/AdminLayout.tsx</code>

6.7 Vérifications phase 2 (testées en vrai)

Élément	Résultat
Compilation backend (JDK 24 $\rightarrow$ <code>release 21</code>) + build frontend (<code>tsc</code> + <code>vite</code>)	✔ 0 erreur ; image Docker reconstruite, conteneur relancé.
Migration <code>V24__notifications_phase2.sql</code>	✔ log : « <i>Migrating schema to version 24 - notifications phase2 ... Successfully applied 1 migration, now at version v24</i> ».
Déduplication / regroupement	✔ ligne pré-existante (<code>dedupe_key</code> = <code>ORDER_PAID:<id></code>) + événement <code>OrderPaid</code> réel de même clé $\rightarrow$ toujours 1 ligne , <code>group_count</code> = 2 , titre/corps rafraîchis (pas de doublon).

Accusé de réception	✔ POST /notifications/{id}/ack → 204 ; la ligne passe read = true, acknowledged = true ; compteur non-lus 2 → 1.
Mise en veille	✔ POST /notifications/{id}/snooze?hours=48 → 204 ; compteur 2 → 1 ; la notification disparaît de la liste jusqu'à échéance.
Préférences par utilisateur	✔ GET /notifications/preferences → [] ; PUT /notifications/preferences/ENROLLMENT_STATUS {inApp:false,email:true} → 200 ; relecture cohérente.
Écran admin « Alertes »	✔ GET /admin/alerts?severity=ALL&status=all → 3 (WARNING / INFO / CRITICAL) ; severity=CRITICAL → 1.
Seuils configurables	✔ GET /admin/alerts/thresholds → 3 sondes ; PUT ../REPORT_FAILURE {enabled:false>windowHours:12} → 200, persisté (puis rétabli).
Données de test	Nettoyées ; un jeu de démo est en place pour l'essai navigateur (admin : 4 dont une groupée ×3 et 2 alertes ; médecin / partenaire / clients : 1–2 chacun).

7. Ce qui reste

Sujet	Détail
Temps réel SSE	Remplacer le polling 30 s par un flux <code>GET /notifications/stream</code> (<code>EventSource</code>). Changement isolé mais touche le filtre de sécurité (l' <code>EventSource</code> ne peut pas envoyer d'en-tête <code>Authorization</code> → passer le JWT en <code>?token=</code> validé par un filtre) — à faire délibérément.
Web Push / FCM	Notifications navigateur en arrière-plan (hors onglet).
E-mails digest	Résumé quotidien / hebdo au lieu d'un e-mail par événement.
Migration des toasts	~20 pages utilisent encore leur <code>useState</code> + <code><Toast/></code> local — remplacer par <code>useToast().notify(...)</code> au fil de l'eau (le composant d'origine reste en place entre-temps).